

# Professor Availability System

Hudson Watts  
*Undergraduate Student*  
*University of Guelph*  
Guelph, Canada  
wattsh@uoguelph.ca

Kavin Gunasekaran  
*Undergraduate Student*  
*University of Guelph*  
Guelph, Canada  
kgunasek@uoguelph.ca

Maya Trajceska  
*Undergraduate Student*  
*University of Guelph*  
Guelph, Canada  
mtrajces@uoguelph.ca

**Abstract**—This Paper is an approach to develop a professor availability system. In attempts to solve common student issues, this system will both aid professors and students. It uses many modes of communication, and many different sensors while attempting to maintain maximum energy efficiency. Using a one-way communication method, there will be a node receiving inputs in the external environment, while the internal environment node will be sending the states of the professor.

**Index Terms**—WSN - Wireless Sensor Network, SDA - Serial Data Pin, SCL - Serial Clock Pin

## I. INTRODUCTION

This is a detailed document explaining the design and integration of a University/Collage Professor Availability System, aimed at addressing a common challenge faced by students: determining a professor's availability. Students often find it difficult to know whether a professor is in a meeting, out of office, on leave. This issue is effectively addressed using a Wireless Sensor Network (WSN). The system uses two ESP32 nodes for one-way communication, enabling real-time transmission of the professor's availability status based on button inputs. This straightforward yet efficient solution enhances accessibility and minimizes uncertainty for students seeking in-person interactions with their professors.

## II. RELATED WORK AND MOTIVATION

This system was inspired by students at the University of Guelph, who frequently expressed frustration over not knowing their professors' availability. Recognizing this as a common issue, the team decided to develop a solution. Drawing inspiration from the status sliders often found on professors' doors, the team devised a simple yet technologically advanced system to address this challenge. Similar to the room and computer availability system in the Albert A. Thornbrough Building at the University of Guelph, this project adopts a comparable approach by using a Wireless Sensor Network (WSN). While the functionality differs, the system uses an LCD display to dynamically present availability states, offering a practical and efficient method of communication.

## III. SYSTEM DESIGN

The system is composed of two main components: the in-office unit and the external door unit. Using ESP32 modules along with additional components detailed in later sections, a functional and efficient system was developed. The in-office unit features switches that allow the professor to set

their availability status, accompanied by LEDs indicating the selected state. Additionally, the professor has access to a web server that displays the output from a camera installed on the door. The external door unit includes a camera, an LCD, and a motion sensor. When motion is detected, the LCD activates and displays the professor's current availability status, providing a seamless and automated solution.

### A. In-Office Component

Inside the professor's office, a button pad allows the professor to select their availability status. There are four possible states: in office, out of office, in a meeting, or on leave. An ESP32 module is integrated into the system to facilitate communication with the ESP32 module on the door unit. When a button is pressed, the in-office ESP32 reads the input and transmits the selected state to the ESP32 at the door. The door ESP32 then updates the LCD to display the professor's current status.

A one-way communication system is used between the two nodes, as this design is energy-efficient and sufficient for the application. Since the door node only needs to display the selected state and does not need to send feedback to the office node, two-way communication is unnecessary and would consume additional power without added benefit.

### B. Hardware description of In-Office Component

Components in the system:

- Four Red LED's
- Four 1k Ohm resistors
- Four 1.5K Ohm resistors
- Four Button Switches
- One ESP32
- One Arduino Nano
- One Battery Pack
- One Switch
- charging unit
- voltage booster

To complete the task, the in-office device uses the components described above. Firstly, to communicate between nodes, the group chose an ESP-32. This choice was made because the ESP-32 allows for quick and efficient peer-to-peer communication through Wi-Fi. The ESP-32 is also equipped with UART to communicate with the Arduino.

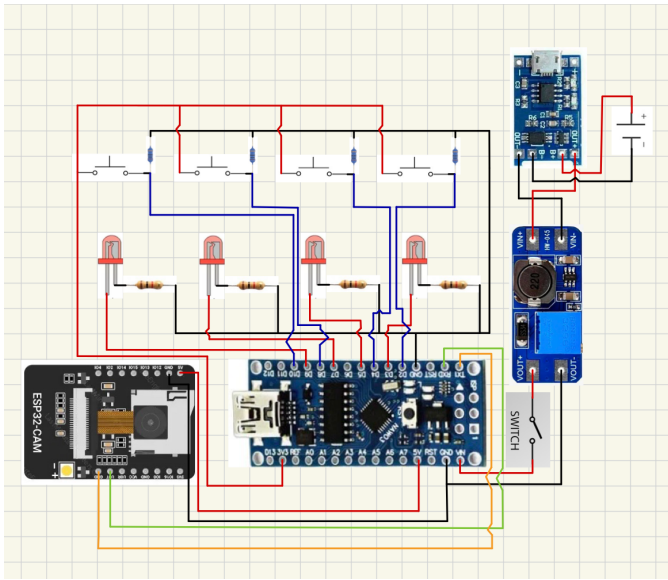


Fig. 1. Schematic of in-office hardware flow

The next component the group decided on was an Arduino Nano, selected for its compact size, large number of addressable pins, and simplicity in communicating with the ESP-32 through UART.

Button switches were used as simple peripherals for interaction because they only have two terminals. The first terminal on each switch is connected to the 3.3V pin, while the second is connected to a digital read pin on the Arduino and grounded through a 1.5kOhm resistor. When the digital pin reads 3.3V, it signals the Arduino that the switch has been pressed. When the button is released, the read pin is disconnected from power and pulled back to ground by the Arduino through the resistor.

LEDs were chosen as a simple way to display the current state of the device. The positive terminal of each LED is powered by digital output pins on the Arduino when the corresponding switch reads a high of 3.3V. The LEDs are grounded through a 1kOhm resistor to reduce the current. When a button is pressed, the information associated with the switch is also transmitted to the ESP-32 through the TX line of the UART connection. This data is then sent to the door device to be displayed.

The group chose a 3.4V battery to power all the devices. However, since the Arduino requires 7V to 12V, the battery will need to be recharged periodically. To solve this, the battery is connected to the input of a charging unit so it can be charged via a port. The input is also connected to the output terminals to continue providing 3.4V. The output of the charging port is connected to the Vin of a voltage booster, enabling the output to produce 7V to power the Vin of the Arduino.

### C. Software description of In-Office Component

The software design for the in-office component of the system focused on two primary objectives: efficient communication and power management. The software for the Arduino

Nano was developed to manage state tracking and message transmission. The Arduino's primary role is to detect button presses, update the corresponding state, and communicate this state to the ESP32. A state machine was implemented to ensure that the Arduino holds the current state until another switch is pressed. This approach prevents redundant messages from being sent and guarantees that a valid status message is always displayed. To provide immediate feedback to the user, the Arduino is connected to four LEDs, each corresponding to a specific state, which light up to reflect the last button pressed.

The Arduino communicates with the ESP32 via its UART Tx and Rx pins. Messages are transmitted using the Serial Monitor, which allows strings representing the states to be sent directly from the Arduino to the ESP32.

The ESP32 serves as the communication hub, receiving messages from the Arduino and utilizing them in two ways. First, it transmits the state information to the door-side ESP32 using ESP-NOW, a lightweight and efficient wireless communication protocol. Second, ESP32 hosts a web server at its assigned IP address to display the current status. This web server uses HTML to create a user-friendly interface, where users can view the professor's availability in real-time. Additionally, the web server incorporates a live camera feed from the door-side ESP32, offering enhanced functionality for monitoring.

To ensure power efficiency, the ESP32 monitors client connections to its web server. If no clients are connected, the ESP32 enters sleep mode, significantly reducing power consumption. This feature minimizes unnecessary energy usage while maintaining system readiness for new connections. The integration of the web server was also pivotal in debugging communication protocols, as it provided a visual confirmation of the system's operation during development.

### D. Door Component

The door component of this system plays a critical role for students, as it provides a visible display of the professor's current availability status. This unit includes a motion sensor connected to the LCD display. When the motion sensor detects movement, it activates the LCD's backlight to ensure visibility.

An ESP32 module in the door unit receives input signals from the ESP32 located in the professor's office. Based on these inputs, the LCD displays the availability state selected by the professor. Additionally, the system features a camera that records whenever someone approaches the door. The professor can access these recordings via a web server linked to the motion sensor, allowing them to see who has been at their door.

### E. Hardware Description of Door Component

The hardware of this system consists of many different sensors to complete the necessary set up. Extensive calculations were performed to find the correct voltage and current ratings of this electrical system.

Components in the system:

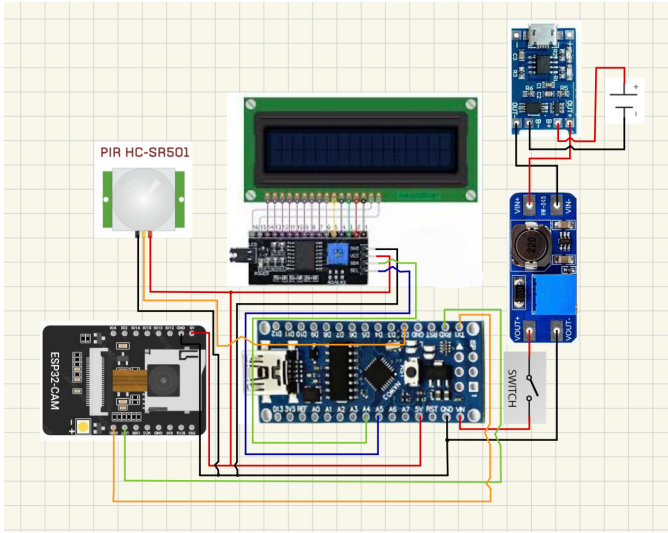


Fig. 2. Schematic of the Door hardware system setup

- PIR Motion Sensor
- 16x2 LCD Display
- ESP32 Camera
- Arduino Nano

As shown in Figure 2, a complete hardware diagram illustrates the connections used in this system. The PIR motion sensor is connected to the Arduino Nano and is programmed to detect motion up to approximately four meters. The PIR motion sensor has three pins: Voltage Input, Ground, and Output (Out). The Voltage Input requires 5V, and the Output pin is connected to digital pin 13 on the Arduino Nano.

The LCD display includes pins for Ground, Voltage Input, SDA, and SCL. The Voltage Input is connected to the Arduino's 5V supply. The SDA pin is connected to Analog Pin 4, and the SCL pin is connected to Analog Pin 5 on the Arduino Nano. The motion sensor and LCD are linked so that motion detected by the sensor controls the LCD's backlight. When motion is detected, the backlight turns on, and the LCD displays the professor's current state. When no motion is detected, the backlight turns off to conserve energy.

The ESP32 module with the camera serves as the wireless sensor node, enabling communication with the in-office node (home base). The ESP32 has pins for Voltage Input, Ground, UOA, and UOT. The module requires a 5V input, with the UOA pin connected to the Arduino Nano's TX pin and the UOT pin connected to the RX pin.

All of these components work together seamlessly to create a functional and efficient system. The hardware setup combines straightforward sensor networks with more complex software implementations to achieve the desired functionality.

#### F. Software Description of Door Component

While the hardware components are relatively straightforward compared to the software, the co-design of both is critical to achieving a functional and efficient system. The integration

of hardware and software ensures seamless communication and operation across the system's various modules.

Before implementing the finalized design, each hardware component was tested to ensure compatibility with its corresponding software. The first stage focused on the integration of the motion sensor and LCD display. Using the Arduino IDE, the motion sensor's input was read to control the LCD. When the motion sensor detected movement (a HIGH signal), the LCD backlight would turn on and display the message "Motion Detected!". Otherwise, the backlight would turn off, and the LCD would display "No Motion Detected." After successfully programming the motion sensor and LCD interaction, attention shifted to the ESP32 camera.

The ESP32 camera was initially configured using a sample program provided in the Arduino IDE. This example served as a foundation for enabling the camera and setting up its basic functionality. A web server was created to stream live video from the camera, allowing the professor to view real-time footage of their door. This web server was developed using a combination of Arduino code and HTML, enabling the camera to capture and display video feed dynamically.

The final and most critical stage was the integration of the wireless sensor network to facilitate communication between the in-office ESP32 node (home base) and the door ESP32 camera. The software for this integration was carefully designed to ensure reliable one-way communication. The home base ESP32 sends unique signals corresponding to the professor's selected availability state. These signals are transmitted wirelessly to the door ESP32, which interprets them and updates the LCD to reflect the appropriate status. Simultaneously, the ESP32 camera streams live footage, providing an additional layer of interactivity and functionality.

This communication was achieved using Wi-Fi protocols embedded within the ESP32 modules. The in-office node uses predefined GPIO pins to send encoded messages that are decoded by the door node. The door node, in turn, triggers actions such as updating the LCD backlight and availability display, as well as maintaining the camera stream for the professor's web server. This integration ensures real-time synchronization between the two nodes, forming a cohesive and responsive system.

#### IV. COMMUNICATION UNIT

The communication protocol for the system centers around the use of ESP-NOW, a lightweight and efficient wireless protocol designed for peer-to-peer communication. The software for the sender ESP32 focuses on two primary objectives: reliable status transmission and seamless integration with the receiver ESP32. To achieve this, the sender registers the receiver's unique MAC address, establishing a direct communication link. Similarly, the receiver registers the sender's MAC address to ensure it only processes messages from a trusted source. This approach eliminates the need for a Wi-Fi router or access point, streamlining the communication process.

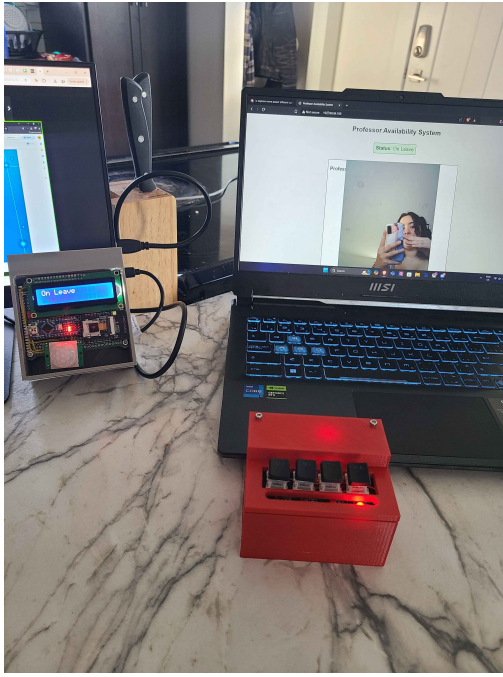


Fig. 3. Full system with LCD display On, motion detected

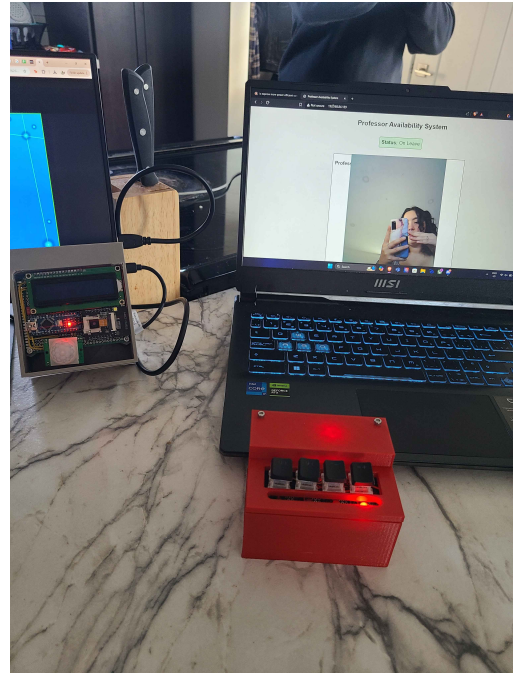


Fig. 4. Full system, with LCD display off, no motion detected

Messages such as "In Office" or "Out of Office" are formatted as structured data and transmitted as ESP-NOW packets directly to the receiver. To ensure reliability, the sender implements a callback function to confirm the delivery status of each packet, while the receiver employs a corresponding callback to handle incoming data, parse the message, and perform actions like toggling GPIO pins to reflect the status. This efficient data exchange is made possible by ESP-NOW's ability to transmit small packets (up to 250 bytes) quickly and with minimal overhead, making it ideal for real-time updates in low-power systems.

The system also leverages ESP-NOW's capability for *one-way communication*, where the sender transmits updates without expecting acknowledgments or responses from the receiver. This design simplifies the protocol's implementation and reduces latency, ensuring swift updates while conserving power. Additionally, the hybrid approach combines ESP-NOW with traditional Wi-Fi functionality, where the sender's ESP32 serves as a web server to display status updates and embed a live video feed streamed from the receiver ESP32. This dual-protocol setup takes advantage of ESP-NOW's efficiency for localized communication and Wi-Fi's broader capabilities for hosting a user-friendly web interface. Together, these protocols provide a robust, low-power, and feature-rich communication framework tailored to the system's requirements.

## V. DATA ANALYSIS AND RESULTS

When receiving data from the desk component, the door device utilizes 4 digital pins and switch cases to turn a 5V high into usable data to display the state of the state of the office. Each of the 4 pins used will correlate to a state in a

switch case. for example if switch 1 is pressed it will send a digital high to pin I04 which will be translated by the Arduino to the message "out of office" will be displayed on the LCD. When putting this concept into practice the results were proved to be successful a individual button pressed updates the LCD in real time to four unique messages made by the prof.

## VI. ERRORS

After several weeks of development, numerous challenges and errors were encountered during the creation of this system. This section outlines the hardware and software issues faced throughout the process and how they were addressed.

### A. Hardware Errors

While planning the hardware requirements was relatively straightforward, careful consideration was given to ensuring all necessary calculations were performed accurately. During testing, it became apparent that the Arduino Nano was not receiving sufficient voltage. The system required more voltage than the battery could provide. To resolve this issue, a step-up module was added to the circuit, boosting the voltage supply from 3.3V to approximately 10V, ensuring reliable operation.

Another issue encountered involved pin 13 on the Arduino Nano, which was not functioning as expected. It was suspected that the pin might have been previously damaged. To address this problem, the output pin for the PIR motion sensor was reassigned to a different, functional pin on the Nano. This adjustment allowed the motion sensor to work properly without compromising the system's performance.



## B. Software Errors

The primary software issue encountered was the LCD display failing to correctly read the button inputs. This posed a significant challenge since the LCD is a critical component of the system. Resolving this issue was both complex and time-consuming. When multiple pieces of software are integrated, debugging becomes increasingly difficult as pinpointing the root cause of an error is not always straightforward.

Initially, the issue was suspected to be within the LCD code, specifically related to clearing the display. The problem stemmed from the LCD continuously appending new values to the existing display instead of overwriting them. The solution involved modifying the display code to clear the screen before updating it with new values. A missing line in the program's loop was identified as the root cause, which prevented the LCD from clearing each time a new signal was passed. Adding this line ensured that only the current values were displayed, resolving the issue.

## VII. CONCLUSION

In conclusion the overall system was successful and it is efficient for both students and professors. After intensive debugging and planning there were conclusions that were drawn that although the ESP32 nodes were simple wireless sensor networks that were used, there are better sensors in which can help improve the system,

## REFERENCES

- [1] Ruiz Brothers Contributors:Liz Clark, "MX Midi Guitar," Adafruit Learning System, <https://learn.adafruit.com/mx-midi-guitar/wiring-mx-switches> (accessed Nov. 15, 2024).
- [2] Alain et al., "Esp32 deep sleep with Arduino Ide and wake up sources," Random Nerd Tutorials, <https://randomnerdtutorials.com/esp32-deep-sleep-arduino-ide-wake-up-sources/> (accessed Nov. 15, 2024).
- [3] D. Amos et al., "ESP32-cam video streaming web server (works with home assistant)," Random Nerd Tutorials, <https://randomnerdtutorials.com/esp32-cam-video-streaming-web-server-camera-home-assistant/> (accessed Nov. 15, 2024).
- [4] Jd et al., "Getting started with ESP-now (ESP32 with Arduino IDE)," Random Nerd Tutorials, <https://randomnerdtutorials.com/esp-now-esp32-arduino-ide/> (accessed Nov. 15, 2024).
- [5] Espressif Systems, ESP-NOW, [https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp\\_now.html](https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_now.html) (accessed Nov. 15, 2024).
- [6] DroneBot Workshop, "ESP now - peer to peer ESP32 communications," DroneBot Workshop, <https://dronebotworkshop.com/esp-now/> (accessed Nov. 15, 2024).
- [7] justAuser et al., "ESP32-cam ai-thinker Pinout Guide: Gpios usage explained," Random Nerd Tutorials, <https://randomnerdtutorials.com/esp32-cam-ai-thinker-pinout/> (accessed Nov. 15, 2024).
- [8] H. Siebeneicher, Device to Device Communication with ESP-NOW, <https://docs.arduino.cc/tutorials/nano-esp32/esp-now/> (accessed Nov. 15, 2024).
- [9] KEB05, "How to use a PIR motion sensor with Arduino," How to Use a PIR Motion Sensor With Arduino, <https://www.instructables.com/How-to-Use-a-PIR-Motion-Sensor-With-Arduino/> (accessed Nov. 15, 2024).
- [10] Utsource, "Getting started with ESP32 CAM: Streaming video using ESP cam over WIFI: ESP32 security camera project," Getting Started With ESP32 CAM — Streaming Video Using ESP CAM Over Wifi — ESP32 Security Camera Project, <https://www.instructables.com/Getting-Started-With-ESP32-CAM-Streaming-Video-Usi/> (accessed Oct. 25, 2024).